

Reviewer Pack — Minimal Rank of Tropical Curves Bounds XOR Circuit Depth

Entry 09519783948e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 14:29:53 UTC

Minimal Rank of Tropical Curves Bounds XOR Circuit Depth

Entry ID: 09519783948e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 14:29:53 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Circuit Complexity

Statement:

[‘For every XOR 3-CNF formula F with n variables and m clauses, the minimal rank of a tropical curve representing F is upper-bounded by a function $f(n) = \Theta(\log n)$. Specifically, there exists a constant C such that for all instances, the minimal rank $r(F) \leq C \log n$.’, ‘Equivalently, for every XOR 3-CNF formula F , if its associated tropical curve has minimal rank greater than some threshold $\gamma \log n$, then F requires an XOR circuit of depth at least $(1/\gamma)$.’, ‘This conjecture can be falsified by finding an XOR 3-CNF formula whose associated tropical curve has a minimal rank exceeding $\gamma \log n$ while being refuted by an XOR circuit of depth less than $(1/\gamma)$.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a bridge between algebraic curves and the analysis of arithmetic operations in computer science. The proposed conjecture aims to leverage this connection to bound the complexity of XOR circuits, which is central to understanding computational complexity.’, ‘The minimal rank of tropical curves has been studied in various contexts within tropical geometry but not yet explicitly connected to circuit complexity. This bridge might reveal new insights into both fields by uncovering a deep structural relationship between arithmetic operations and geometric properties.’, ‘By demonstrating such a connection, the conjecture could potentially lead to new algorithms or lower bounds for XOR circuits.’]

Taxonomy category: TROPICAL_GEOMETRY_XOR_CIRCUIT_DEPTH (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 955baab4c30d9805

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given XOR 3-CNF formula F with n variables, if its minimal rank $r(F)$ exceeds $\gamma \log n$ (where γ is a constant), then an XOR circuit with depth less than $1/\gamma$ must be constructed to refute the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "Boolean circuit complexity" AND "minimal rank" AND "XOR circuit depth" - "XOR 3-CNF formula" AND minimal rank tropical curve AND upper bound" - "tropical curve" AND Boolean circuit AND XOR circuit depth threshold"

Top relevant hits considered: - [<http://arxiv.org/abs/2204.02599v1>] Local theory of functions on tropical curves in $\mathbb{R}^n$ - [<http://arxiv.org/abs/1507.05818v2>] The Scaling Site - [<http://arxiv.org/abs/1603.03191v1>] Geometry of the scaling site

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        factor = -A[i][i] / A[i][i]
        for j in range(i+1, n):
            A[j][i] = 0
            for k in range(i+1, n):
                A[j][k] += factor * A[i][k]

    # Back-substitute to find the rank
```

```

rank = n
for i in range(n-1, -1, -1):
    if all(A[i][j] == 0 for j in range(i+1, n)):
        rank -= 1
    else:
        break
return rank

def rank(matrix):
    A = [row[:] for row in matrix]
    return gaussian_elimination(A)

def generate_xor_3cnf(n, m):
    variables = [f"x{i}" for i in range(1, n+1)]
    clauses = []
    for _ in range(m):
        clause = random.sample(variables + ['~' + v for v in variables], 3)
        clauses.append(clause)
    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Parameters
    n_values = [5, 10, 15, 20, 30, 40]
    gamma = Fraction(1, 2) # Example constant for demonstration

    results = []
    for n in n_values:
        instances_tested = 0
        conjecture_holds = True
        counterexample = ""

        for _ in range(30):
            clauses = generate_xor_3cnf(n, int(1.5 * n))
            matrix = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

            minimal_rank = rank(matrix)
            instances_tested += 1

            if minimal_rank > gamma * math.log2(n):
                # Construct XOR circuit
                depth = int(1 / gamma) + 1
                conjecture_holds = False
                counterexample = f"n={n}, min_rank={minimal_rank}, depth={depth}"
                break

        results.append({
            "metric_name": "min_rank",
            "metric_value": minimal_rank,
            "instances_tested": instances_tested,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        })

    mean_metric = sum(result["metric_value"] for result in results) / len(results)

```

```

std_metric = math.sqrt(sum((result["metric_value"] - mean_metric) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "mean_metric": mean_metric,
    "std_metric": std_metric,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric = sum(result["mean_metric"] for result in results) / len(results)
    std_metric = math.sqrt(sum((result["mean_metric"] - mean_metric) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["support_fraction"] == 1) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif any(result["conjecture_holds"] is False for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{first_failing_seed}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1652dcbd.py", line 108, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1652dcbd.py", line 73, in run_trial
    minimal_rank = rank(matrix)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1652dcbd.py", line 46, in rank
    return gaussian_elimination(A)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1652dcbd.py", line 29, in gaussian_elimination
    factor = -A[i][i] / A[i][i]
              ~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify the conjecture's falsification or support. | next: Re-run the test without errors to determine if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13226
2	preregistration	ollama_remote	glm4:latest	0	0	6037
3	novelty	ollama_remote	glm4:latest	0	0	4886
4	novelty	ollama_remote	glm4:latest	0	0	5724
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13861
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7773
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10339
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12025
9	judge	ollama_remote	glm4:latest	0	0	8275

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 82146 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/09519783948e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/09519783948e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/09519783948e.tar.gz (if generated)